Reading Vivado Reports

What every field in the synthesis and implementation reports means, how to check S1–S5, and how to read the files in `report/builds/`. Examples use our first real build, B01 (`top_basys3`, 4 Nov 2026), which failed timing; §9 shows how the final build B05 passes every check.

1. From RTL to bitstream
2. The build folder and `build_info.txt`
3. Utilization report
4. Timing summary: WNS, TNS, WHS, …
5. Reading one timing path
6. `check_timing`: are the constraints complete?
7. Synthesis warnings (S1)
8. DRC report (S5)
9. The S1–S5 checklist
10. When timing fails: what to try
11. Glossary

1 · From RTL to bitstream

Verilog + .mem + XDC → Synthesis → opt_design → place_design → route_design → write_bitstream → .bit

Step	What it does	What can go wrong here
Synthesis	Turns Verilog into a netlist of FPGA primitives: LUTs, flip-flops, carry chains (CARRY4), block RAMs (RAMB36/18), DSP48 slices. Loads `$readmemh` files into ROM contents.	Latches, multi-driven signals, logic removed because nothing uses it, a `.mem` file not found (ROM becomes all zero).
opt_design	Removes constant and duplicate logic.	Logic you expected disappears because its output is never used.
place_design	Chooses a physical location (slice, BRAM site, DSP site) for every primitive.	Over-full device (not our problem: we use about 3% of LUTs).
route_design	Connects the placed primitives with wires. Real wire delays are only known now.	Timing fails: a path takes longer than one clock period.
write_bitstream	Runs the final DRC (design rule check) and writes the `.bit` file.	Stops on DRC errors, for example a port with no pin or no IOSTANDARD in the XDC.

> **Synthesis estimates, implementation measures.** Synthesis reports (`*_synth_*.txt`) guess wire delays. Only the post-route reports (the ones in `report/builds/`) count for S4 and for the final report.

2 · The build folder and `build_info.txt`

Each run of `build_bitstream.tcl` creates `report/builds/Bnn_<commit>/` :

File	Contents	Used for
`build_info.txt`	One-page summary (below)	The row in `implementation_log.md`
`utilization.txt`	Resources used by the whole design (§3)	S2, S3, resource table

`utilization_hier.txt`	The same, split per module instance	Which module uses what; S2 per memory
`timing.txt`	Timing summary, `check_timing` and the worst path per clock (§4–6)	S4, Fmax
`critical_paths.txt`	The 10 worst setup paths	Deciding what to fix
`synth_warnings.txt`	Every WARNING / CRITICAL WARNING / ERROR line of synthesis	S1 (§7)
`drc.txt`	Design rule check of the routed design	S5 (§8)

```
build    B01                       build number, counts up
commit   2583f6a (day5/synthesis)  the exact source that was built
dirty    no                        "yes" = uncommitted hw/ or mem/ changes
vivado   2026.1
top      top_basys3
generic  (none)                    empty = USE_STUB 0 = real CNN core
WNS      -0.259 ns                 setup slack, must be >= 0     (§4)
TNS      -2.059 ns                 sum of all negative slack, must be 0
WHS      0.096 ns                  hold slack, must be >= 0
LUT 652 · FF 405 · BRAM 12.5 tiles · DSP 0                      (§3)
```

> A `_dirty` build can't be rebuilt from git. Commit first, then build, so that anyone can reproduce the numbers in the report.

3 · Utilization report

The columns

```
| Site Type        | Used | Fixed | Prohibited | Available | Util% |
| Slice LUTs       |  652 |     0 |          0 |     20800 |  3.13 |
```

Column	Meaning
Used	How many the design uses. **This is the number for the report.**
Fixed	How many were placed by hand with location constraints. Always 0 for us except I/O pins.
Prohibited	Sites that can't be used. 0 for us.
Available	Total on the XC7A35T.
Util%	Used ÷ Available.

The rows that matter

Row	What it is	B01	What to look for
Slice LUTs	6-input look-up tables: all combinational logic (adders, muxes, comparators, FSM next-state logic)	652 / 20,800	Large jumps between builds. A LUT multiplier shows up here instead of in DSPs.
LUT as Logic	LUTs used as logic	652	

LUT as Memory	LUTs used as small RAM/ROM ("distributed RAM")	0	Our small ROMs (conv weights, biases, labels) can appear here or be folded into logic.
Slice Registers	Flip-flops (FF): every `reg` assigned in `always @(posedge clk)`	405 / 41,600	Should match the counters, state and data registers you wrote.
F7 / F8 Muxes	Dedicated wide multiplexers that combine LUTs	0	Informational.
Block RAM Tile	36 Kb block RAMs. One RAMB36 = 1 tile, one RAMB18 = 0.5 tile	12.5 / 50 (25%)	Target ≤ 30% (D19). Should be: image ROM, fmap RAM, pool RAM, FC weight ROM (S2).
RAMB36 / RAMB18	The split into full and half blocks	12 + 1	One RAMB36 holds 4,096 × 8 bits (4 KB of our data); one RAMB18 holds 2,048 × 8.
DSPs	DSP48E1 slices: hard 25×18 multiplier + 48-bit adder	0 / 90	S3 expects 2 (one per MAC). 0 means the multiplies were built from LUTs and carry chains, which is slower.
Bonded IOB	Package pins used	37 / 106	clk + 16 SW + 2 buttons + 16 LED + 7-seg… should match the ports of `top_basys3`.
BUFGCTRL	Global clock buffers	1 / 32	Exactly 1 (the 100 MHz clock). More means something is being used as a clock by mistake.

> **Primitives section.** Further down, the report lists primitives by name: `FDRE` (flip-flop with synchronous reset), `LUT1…LUT6`, `CARRY4` (4-bit carry chain for adders), `RAMB36E1`, `DSP48E1`, `IBUF` / `OBUF` (pin buffers). Use it to check which primitives were inferred.

Hierarchical utilization (`utilization_hier.txt`)

One row per instance, indented by hierarchy, for example `top_basys3 → gen_core.u_core → u_fc → fc_w_rom`. Use it to say "the FC engine uses N LUTs" in the final report, and to confirm each memory landed in block RAM (S2).

---

4 · Timing summary: WNS, TNS, WHS, …

Setup and hold in one picture

Every path starts at a register (or BRAM) clocked by one edge and ends at a register clocked by the next edge. At 100 MHz there are 10 ns between edges.

- **Setup**: the data must *arrive* before the next edge (minus a small setup time). If the logic between registers is too slow, setup fails. Fixed by less logic per cycle.
- **Hold**: the data must *stay* stable a little after the edge. Fails when a path is too fast, usually because of clock skew. Vivado normally fixes hold by itself during routing.

```
    WNS(ns)      TNS(ns)  TNS Failing Endpoints  TNS Total Endpoints      WHS(ns)      THS(ns)  THS
Failing Endpoints ...  WPWS(ns)  TPWS(ns)
     -0.259       -2.059                     16                 1238        0.096        0.000
0 ...     4.500     0.000
```

Field	Meaning	Pass when	B01
WNS	**Worst Negative Slack** (setup): the slack of the single worst path. Slack = required time − arrival time.	≥ 0	**−0.259**
TNS	**Total Negative Slack**: sum of the negative slacks of all failing endpoints. Shows how big the problem is.	= 0	**−2.059**
TNS Failing Endpoints	How many register inputs fail setup.	0	**16 (bits of `accum`)**
TNS Total Endpoints	How many endpoints were checked.	—	1,238
WHS	**Worst Hold Slack**.	≥ 0	**+0.096**
THS	**Total Hold Slack** (negative sum).	= 0	**0**
WPWS / TPWS	**Pulse Width Slack**: whether the clock's high and low times are long enough for each primitive (BRAM and DSP have minimum pulse widths).	≥ 0 / = 0	**4.5 / 0**

Fmax

The fastest clock the design would pass at: `Fmax = 1 / (T − WNS)`. B01: 1 / (10 ns + 0.259 ns) = **97.5 MHz**. When WNS ≥ 0 the design passes at 100 MHz, and Fmax = 1 / (10 − WNS) is higher than 100 MHz.

> **"It works on the board" is not a pass.** Delays in the report are for the slowest chip at the worst temperature and voltage ("Slow Process Corner"). A failing design can work on your desk and fail on a warmer day or another board. S4 requires WNS ≥ 0.

Clock summary and intra-clock table

Further down: `sys_clk_pin` with period 10.000 ns, waveform {0.000 5.000} (rising at 0, falling at 5). This comes from `create_clock` in `basys3.xdc`. The "Intra Clock Table" repeats WNS/TNS/WHS for paths inside that one clock. We have one clock, so it equals the summary.

5 · Reading one timing path

This is B01's worst path (from `critical_paths.txt`), shortened:

```
Slack (VIOLATED) :        -0.259ns  (required time - arrival time)
  Source:                 gen_core.u_core/u_fc/fc_w_rom/data_reg_0/CLKARDCLK   (RAMB36E1)
  Destination:            gen_core.u_core/u_fc/accum_reg[31]/D                 (FDRE)
  Path Group:             sys_clk_pin
  Path Type:              Setup (Max at Slow Process Corner)
  Requirement:            10.000ns  (sys_clk_pin rise@10.000ns - sys_clk_pin rise@0.000ns)
  Data Path Delay:        10.186ns  (logic 6.205ns (60.917%)  route 3.981ns (39.083%))
  Logic Levels:           14  (CARRY4=9 LUT2=1 LUT3=1 LUT6=3)
  Clock Path Skew:        -0.068ns (DCD - SCD + CPR)
  Clock Uncertainty:      0.035ns
```

Field	Meaning
Slack (VIOLATED / MET)	Required time − arrival time. Negative = VIOLATED.

Source	Where the path starts: a clocked element. Here the clock pin of the FC weight ROM's block RAM, so the path includes the BRAM's output delay.
Destination	Where it ends: the D input of bit 31 of the FC accumulator. The highest bit is worst because the carry has to ripple through all 32 bits.
Path Group	Paths are grouped by destination clock.
Path Type	**Setup** (max delay) or **Hold** (min delay), and the process corner used (Slow for setup).
Requirement	Time available between launch and capture edge: one period, 10 ns.
Data Path Delay	Total delay from source to destination, split into **logic** (inside LUTs, carry chains, BRAM) and **route** (wires). Mostly logic → too much logic per cycle (pipeline or use a DSP). Mostly route → placement or high fan-out.
Logic Levels	How many primitives the signal passes through. 14 is a lot for 100 MHz; under about 8 is comfortable. Here 9 are CARRY4: the multiplier and the 32-bit adder built from carry chains.
Clock Path Skew	Difference in clock arrival between destination and source. **DCD** = destination clock delay, **SCD** = source clock delay, **CPR** = clock pessimism removal (a correction for the shared part of the clock tree). Small (±0.1 ns) is normal.
Clock Uncertainty	Safety margin for jitter. **TSJ** = total system jitter, **TIJ** = input jitter (from `set_input_jitter`, 0 for us), **DJ** = discrete jitter (from clock generators like an MMCM, 0 for us), **PE** = phase error.

The delay table

```
Location        Delay type                      Incr(ns)  Path(ns)   Netlist Resource(s)
RAMB36_X2Y5     RAMB36E1 (Prop_..._DOADO[1])       2.454     7.575 r  .../fc_w_rom/data_reg_0/DOADO[1]
                net (fo=18, routed)                1.445     9.020    .../pool_ram/out[1]
SLICE_X58Y21    LUT6 (Prop_lut6_I3_O)              0.124     9.144 r  .../product__0_carry__0_i_4__0/O
SLICE_X58Y19    CARRY4 (Prop_carry4_S[0]_CO[3])    0.532    10.369 r  .../u_fc/product__0_carry__0/CO[3]
  ...
SLICE_X57Y26    FDRE                                                r  .../u_fc/accum_reg[31]/D
```

- **Location**: the physical site (BRAM column/row, slice X/Y).
- **Delay type**: `Prop_…` = delay through a primitive from one pin to another; `net (fo=18, routed)` = a wire with fan-out 18 (it drives 18 inputs).
- **Incr** = this step's delay; **Path** = running total since the launch clock edge (the clock tree delay, about 5.1 ns, is included at the top).
- **r / f** = rising or falling transition.
- Names like `product__0_carry` and `accum0_carry` come from your Verilog signal names (`product`, `accum`), so you can map the path back to the source code.

The bottom part repeats the clock path for the capture edge and ends with `required time 15.048`, `arrival time -15.307`, `slack -0.259`.

> **Reading B01 in one sentence:** in one clock cycle the FC engine reads a weight from BRAM (2.5 ns), multiplies it by an activation using LUTs and carry chains, and adds it into a 32-bit accumulator. That's 10.19 ns of work in a 10 ns cycle.

6 · `check_timing`: are the constraints complete?

Timing numbers mean nothing if some paths aren't checked. This section of `timing.txt` lists paths Vivado could not analyse.

Check	Meaning	Should be	B01
no_clock	Registers with no clock reaching them	0	0
constant_clock	A clock pin tied to a constant	0	0
pulse_width_clock	Clock pins with only a pulse-width check	0	0
unconstrained_internal_endpoints	Register inputs inside the design not covered by any clock. **The important one.**	0	0
no_input_delay	Input ports without `set_input_delay`	0, or justified	8
no_output_delay	Output ports without `set_output_delay`	0, or justified	23
multiple_clock	A register reached by two clocks	0	0
generated_clocks	Problems with derived clocks	0	0
loops / latch_loops	Combinational loops: an output fed back to its own input without a register. Always a bug.	0	0
partial_input/output_delay	Only min or only max delay set	0	0

> **Our 8 inputs and 23 outputs** are switches, buttons, LEDs and seven-segment pins. Nobody times them against an external chip: buttons and switches go through a 2-flip-flop synchronizer, and LEDs change far slower than a human can see. The usual fix is a `set_false_path` on those ports in `basys3.xdc`, which tells Vivado not to time them. That choice would be recorded in `DESIGN_DECISIONS.md`.

7 · Synthesis warnings (S1)

`synth_warnings.txt` holds every warning line. Message IDs (like `[Synth 8-327]`) can change between Vivado versions, so search by the text. Most warnings are harmless; these are the ones to read carefully:

Message text contains	Meaning	Action
inferring latch	A combinational `always @(*)` doesn't assign a signal on every path (missing `else` or `default`).	**Always fix.** S1 requires 0 latches.
multi-driven net / multiple drivers	Two `always` blocks or `assign`s drive the same signal.	**Always fix.**

is unused and will be removed / Unused sequential element … was removed	A register whose value nobody reads.	Fine for debug signals. If it's a datapath register (accumulator, address counter, ROM data) it means a wiring mistake upstream.
is either unconnected or has no load	A port or signal that goes nowhere.	Check it's intended (e.g. logits not shown on the board are fine).
could not open $readmem data file / file not found	`$readmemh` didn't find a `.mem` file. The ROM is built empty.	**Always fix.** The design will give the same class for every image.
truncated / width mismatch / out of range	An assignment loses bits.	Check each one against the bit-width table in the docs.
CRITICAL WARNING	Something Vivado thinks will probably break the design (e.g. an XDC line that matched no port).	**Read every one.** S1 requires 0.

8 · DRC report (S5)

`drc.txt` lists design rule checks, each with an ID and a severity (Error, Critical Warning, Warning, Advisory). Errors stop `write_bitstream`. Common ones:

ID	Meaning	Action
NSTD-1	A port has no IOSTANDARD in the XDC	Add `set_property IOSTANDARD LVCMOS33`
UCIO-1	A port has no pin location (PACKAGE_PIN)	Add the pin in `basys3.xdc`
CFGBVS-1	Configuration voltage properties not set	Harmless on Basys3; can be silenced with `CFGBVS VCCO` / `CONFIG_VOLTAGE 3.3`
DPIP-1 / DPOP-1	A DSP48 has no input / output pipeline register	Advisory: a hint that adding a register would make the DSP faster
REQP-…	A primitive's connection rules are broken	Read the message; usually a real problem

9 · The S1–S5 checklist

The checks used in this project, with the result of the first build (B01) and the final build (B05). Details per build are in `report/implementation_log.md`.

Check	Requirement	Where to look	B01	B05
S1 Synthesis warnings	No latches, no multi-driven nets, no unexpected removed logic; 0 critical warnings	`synth_warnings.txt` (§7)	not archived	Pass
S2 BRAM inference	`image_rom`, `fmap_ram`, `pool_ram`, `fc_w_rom` in block RAM	`utilization.txt` (RAMB36/18), `utilization_hier.txt` per instance (§3)	Pass: 12.5 tiles	Pass: 12.5 tiles
S3 DSP inference	2 DSP48E1 for the MACs	`utilization.txt` → DSPs (§3)	Fail: 0 DSP	Pass: 2 DSP
S4 Timing	WNS ≥ 0, TNS = 0, no unconstrained paths	`timing.txt` summary (§4) and `check_timing` (§6)	Fail: WNS −0.259 ns	Pass: WNS +0.651 ns, TNS 0 (board I/O unconstrained, see §6)
S5 XDC review	Every port has a pin and IOSTANDARD; no DRC errors	`drc.txt` (§8); bitstream written	not archived	Pass

10 · When timing fails: what to try

Start with the worst path in `critical_paths.txt`. Look at the source, the destination and whether delay is mostly logic or mostly route.

1. **Let the multiply-accumulate use a DSP48.** A DSP does a multiply and add in hard silicon, much faster than LUTs. Vivado sometimes doesn't infer one; `(* use_dsp = "yes" *)` on the module or signal asks it to. Cycle count stays the same.
2. **Add a pipeline register.** Split the long path into two cycles: register the product, then add it next cycle. Faster, but adds latency, so the FSM, cycle count and testbenches must change.
3. **Register the BRAM output.** BRAM output delay is about 2.5 ns. A register right after the BRAM gives the next stage a full cycle. Also adds one cycle of latency.
4. **Reduce fan-out.** If one `net (fo=…)` has a huge fan-out and large route delay, duplicate the driving register.
5. **Don't** lower the clock in the XDC just to make the number pass, and don't ignore a negative WNS because the board seems to work. Both have to be written in `DESIGN_DECISIONS.md` if the team ever chooses them.

> Every fix is a new build: commit, run the tests, build, add the row to `implementation_log.md`. A design choice also goes in `DESIGN_DECISIONS.md`.

11 · Glossary

Term	Meaning
LUT	Look-up table: a 6-input truth table, the basic logic cell
FF / FDRE	Flip-flop; FDRE = D flip-flop with clock enable and synchronous reset
CARRY4	Fast 4-bit carry chain used to build adders and comparators
Slice	A group of 4 LUTs, 8 FFs and one CARRY4
RAMB36 / RAMB18	Block RAM of 36 Kb / 18 Kb; synchronous read (1-cycle latency)
DSP48E1	Hard multiply-accumulate block (25×18 multiplier, 48-bit accumulator)
IOB / IBUF / OBUF	I/O pin block and its input/output buffers
BUFG	Global clock buffer: drives the clock to every register with low skew
Endpoint	Where a timing path ends: the D (or enable/reset) pin of a register or BRAM
Slack	Required time − arrival time; positive = meets timing
Setup / Hold	Data must be stable before (setup) and after (hold) the clock edge
Critical path	The path with the worst slack; it decides Fmax
Logic level	One primitive the signal passes through on a path
Fan-out (fo)	How many inputs one net drives
Skew	Difference in clock arrival time at two registers
Jitter	Random variation of the clock edge time
Process corner	Worst-case chip, voltage and temperature used for analysis (Slow for setup, Fast for hold)
False path	A path told not to be timed (`set_false_path`)
XDC	Xilinx Design Constraints file: pins, IOSTANDARDs, clock definitions
DRC	Design Rule Check: rules the design must meet before a bitstream can be written